

UNIVERSIDAD AMERICANA
Facultad de Ingeniería y Arquitectura



ACTIVIDAD PRACTICA

Practica Integradora de SQL Server

Estudiantes:

- Valeria Carolina Grijalva Arevalo
- Emilio Fernando Meza Ortiz
- Gabriel Alejandro García Angulo
- Gabriel Antonio Rojas Uriarte
- Manuel Joaquín Chamorro Gómez

Docente:

- Jose Duran Garcia

28 de Abril del 2026

Práctica de Laboratorio: Seguridad, Administración y Documentación en SQL Server

Indicador de logro: Implementa mecanismos básicos de administración, seguridad, recuperación y documentación en Microsoft SQL Server, aplicando una secuencia colaborativa de construcción, aseguramiento y optimización de una base de datos mediante evidencias técnicas documentadas.

Caso de estudio: Cadena colaborativa de consultoría para asegurar y documentar una base de datos empresarial

Una empresa farmacéutica requiere una consultoría para evaluar la seguridad, documentación y administración básica de una base de datos operativa.

Cada grupo actuará como una fase especializada de un proyecto de consultoría, donde el producto generado por un grupo servirá como insumo para el siguiente, simulando una cadena de trabajo tipo DevOps y trazabilidad de datos.

Cada equipo entregará scripts, evidencias y conclusiones en PDF.

Secuencia de Trabajo por Grupos Grupo 1: Recorrido por el SGBD y monitoreo Actividades

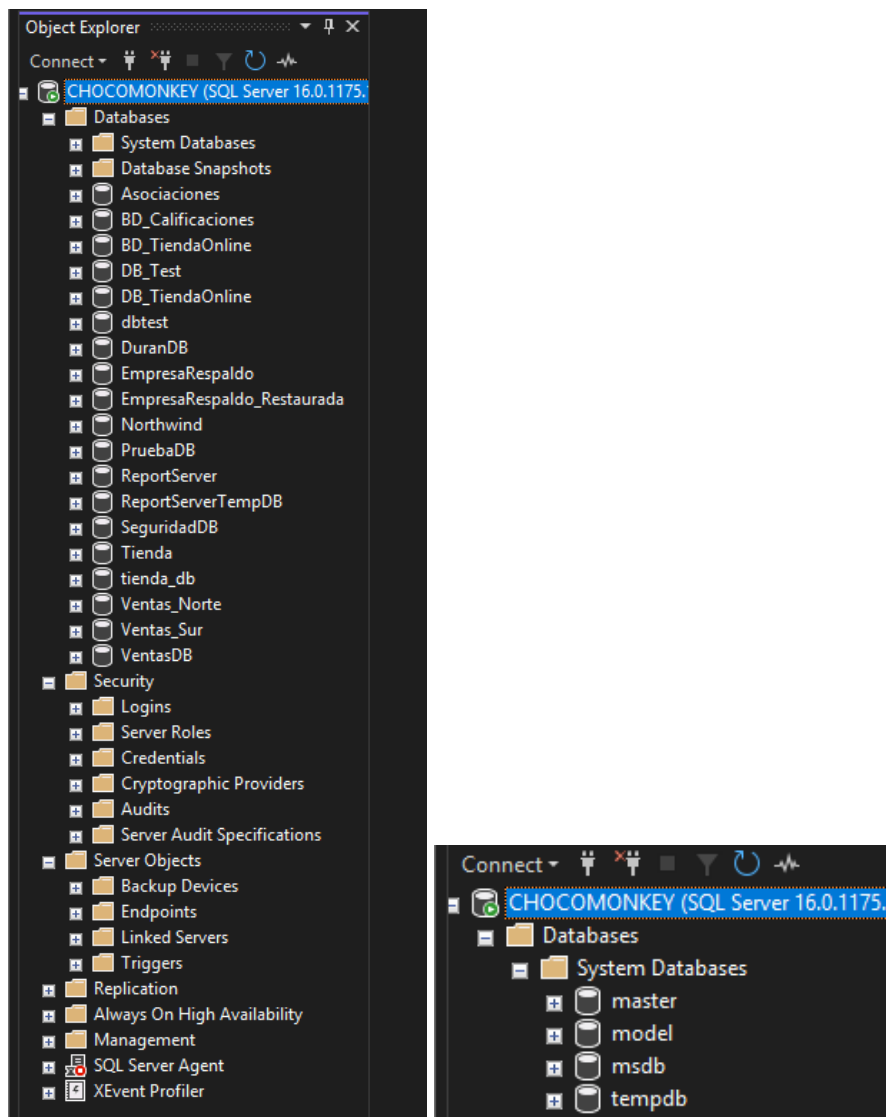
- Explorar el entorno de SQL Server Management Studio.
- Identificar bases de datos del sistema:
 - master
 - model
 - msdb
 - tempdb
- Revisar objetos y configuraciones críticas del servidor.

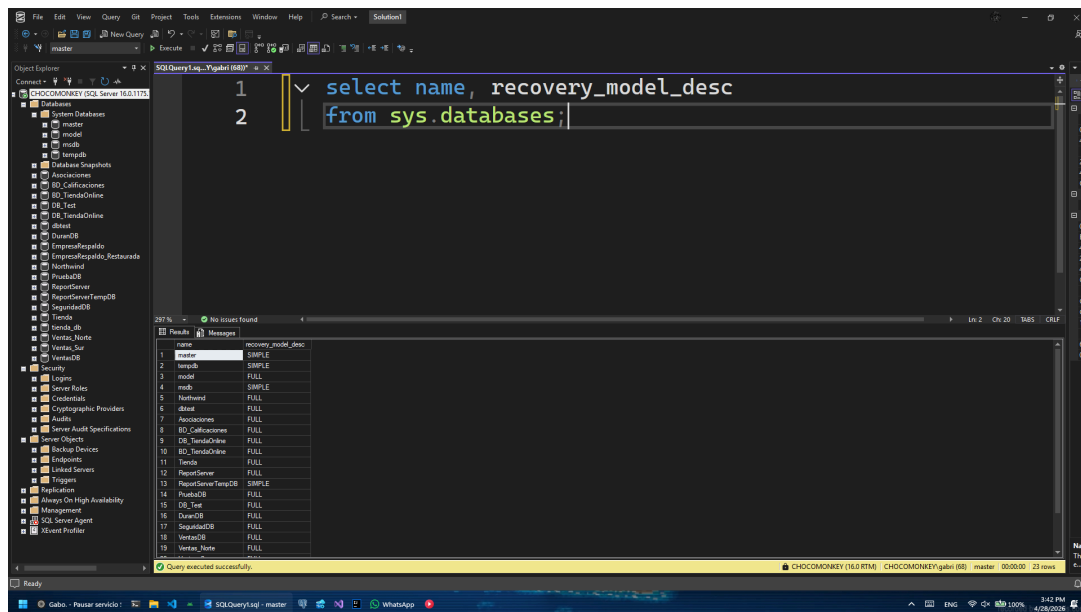
Pautas para orientar solución

1. Usar el Explorador de Objetos para ubicar componentes de seguridad, bases del sistema y configuraciones.
2. Apoyarse en consultas como: `SELECT name, recovery_model_desc`
`FROM sys.databases;`
3. Elaborar un diagnóstico inicial que entregue insumos técnicos al Grupo 2. Cada grupo trabajará sobre la base del grupo anterior.

Entregable

Capturas + diagnóstico del servidor.





En esta fase se realizó una exploración del entorno de SQL Server Management Studio utilizando el **Object Explorer**, con el objetivo de identificar los componentes principales del servidor y comprender su estructura general.

Primero, se accedió al servidor y se revisó la sección de **Databases**, donde se pudieron observar tanto bases de datos del sistema como bases de datos creadas por el usuario. Esto permitió tener una visión general del estado actual del entorno.

Posteriormente, se identificaron las bases de datos del sistema ubicadas dentro de la carpeta **System Databases**, específicamente:

- **master**: contiene la información principal del sistema y configuración del servidor.
- **model**: funciona como plantilla para la creación de nuevas bases de datos.
- **msdb**: almacena información relacionada con tareas administrativas como respaldos y jobs.
- **tempdb**: se utiliza para almacenar datos temporales durante la ejecución de procesos.

Luego, se ejecutó la siguiente consulta:

```
SELECT name, recovery_model_desc
```

```
FROM sys.databases;
```

Esta consulta permitió visualizar todas las bases de datos existentes junto con su modelo de recuperación, lo cual es importante para entender cómo se gestionan los respaldos y la recuperación de datos en cada una.

Además, se revisaron componentes clave del servidor como la sección de **Security (Logins)** y **Server Objects**, con el fin de identificar configuraciones básicas de acceso y administración.

Diagnóstico del servidor:

A partir del recorrido realizado, se puede concluir que el servidor cuenta con una estructura organizada que incluye las bases de datos del sistema necesarias para su funcionamiento, así como múltiples bases de datos de usuario. Se identificaron correctamente los componentes de seguridad y administración, lo que permite continuar con la construcción y configuración de nuevas bases de datos en las siguientes fases del proyecto.

Este análisis inicial proporciona una base clara para que los siguientes grupos puedan trabajar sobre el entorno ya identificado y documentado.

Grupo 2: Construcción de la base de datos Actividades

- Crear base de datos.
- Crear tabla de tres campos.
- Insertar tres registros.
- Ejecutar consultas básicas.

Pautas para orientar solución

1. Diseñar primero estructura y llave primaria.
2. Probar operaciones: `CREATE DATABASE Farmacia;`
`CREATE TABLE Productos(...)`
3. Entregar scripts funcionales al Grupo 3.

Entregable

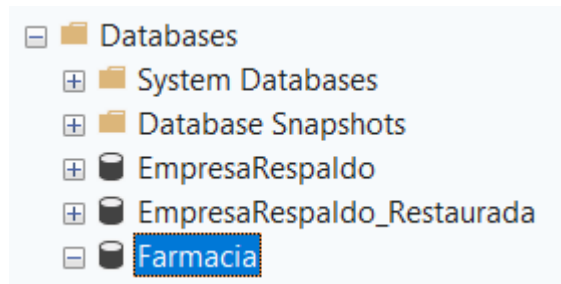
Script SQL + capturas.

Primero se valida si la base de datos FarmaciaDB ya existe. Si no existe, se crea mediante la instrucción `CREATE DATABASE`. Esto evita errores si el script se ejecuta más de una vez.

```

IF NOT EXISTS (
    SELECT name
    FROM sys.databases
    WHERE name = 'Farmacia'
)
BEGIN
    CREATE DATABASE Farmacia;
END;
GO

```

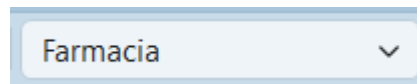


Luego se selecciona la base de datos con USE Farmacia, para asegurar que las siguientes instrucciones se ejecuten dentro de la base correcta.

```

USE Farmacia;
GO

```



Después se crea la tabla Productos, la cual contiene tres campos principales:

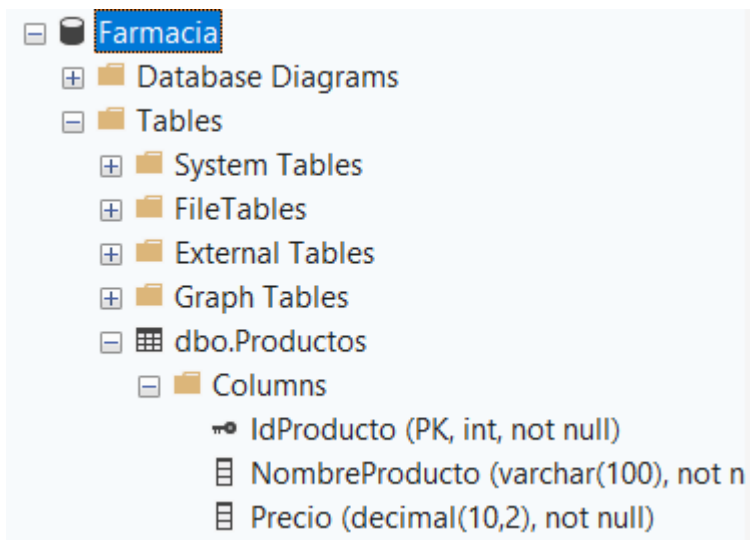
Campo	Tipo de dato	Descripción
IdProducto	INT IDENTITY PRIMARY KEY	Identificador único del producto
NombreProducto	VARCHAR(100)	Nombre del producto farmacéutico
Precio	DECIMAL(10,2)	Precio del producto

La llave primaria es IdProducto, porque permite identificar de manera única cada producto registrado.

```

IF NOT EXISTS (
    SELECT *
    FROM sys.tables
    WHERE name = 'Productos'
)
BEGIN
    CREATE TABLE Productos (
        IdProducto INT IDENTITY(1,1) PRIMARY KEY,
        NombreProducto VARCHAR(100) NOT NULL,
        Precio DECIMAL(10,2) NOT NULL
    );
END;
GO

```



Finalmente, se insertan tres registros y se ejecutan consultas básicas para comprobar que la tabla funciona correctamente.

```

INSERT INTO Productos (NombreProducto, Precio)
VALUES
('Paracetamol 500mg', 35.50),
('Ibuprofeno 400mg', 78.75),
('Amoxicilina 500mg', 120.00);
GO

```

1. Ver todos los productos

```
SELECT *
FROM Productos;
GO
```

	IdProducto	NombreProducto	Precio
1	1	Paracetamol 500mg	35.50
2	2	Ibuprofeno 400mg	78.75
3	3	Amoxicilina 500mg	120.00

2. Buscar productos mayores a cierto precio

```
SELECT IdProducto, NombreProducto, Precio
FROM Productos
WHERE Precio > 50;
GO
```

	IdProducto	NombreProducto	Precio
1	2	Ibuprofeno 400mg	78.75
2	3	Amoxicilina 500mg	120.00

3. Ordenar productos por precio

```
SELECT IdProducto, NombreProducto, Precio
FROM Productos
ORDER BY Precio ASC;
GO
```

	IdProducto	NombreProducto	Precio
1	1	Paracetamol 500mg	35.50
2	2	Ibuprofeno 400mg	78.75
3	3	Amoxicilina 500mg	120.00

Grupo 3: Creación de login y usuario Actividades

- Crear login.
- Crear usuario.
- Asociar usuario con login.

Pautas para orientar solución

1. Diferenciar autenticación y autorización.
2. Guiarse con:

`CREATE LOGIN usuario_lab...`

`CREATE USER usuario_lab`

`FOR LOGIN usuario_lab;`
3. Verificar acceso y pasar configuración al Grupo 4.

Entregable

Scripts + evidencia de autenticación.

La autenticación consiste en validar la identidad de una persona o cuenta que intenta conectarse al servidor. En SQL Server, esto se realiza mediante un LOGIN, el cual permite acceder al servidor.

La autorización define qué permisos tiene ese usuario dentro de una base de datos específica. En SQL Server, esto se maneja mediante un USER, el cual se crea dentro de una base de datos y se asocia a un login existente.

```

-- 1. Crear login a nivel de servidor
CREATE LOGIN usuario_lab
WITH PASSWORD = 'Lab12345*';

-- 2. Usar la base de datos correspondiente
USE Farmacia;
GO

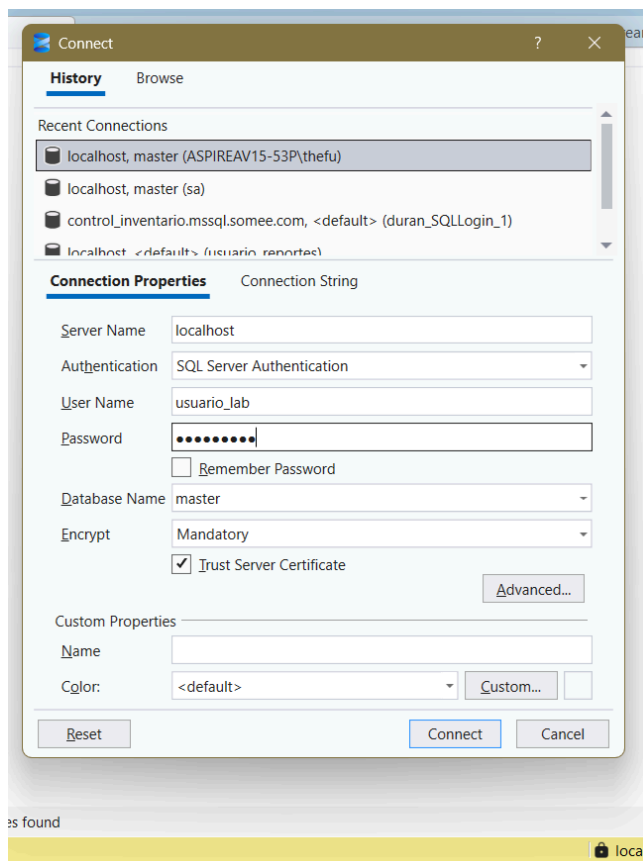
-- 3. Crear usuario dentro de la base de datos
CREATE USER usuario_farmacia
FOR LOGIN usuario_lab;
GO

-- 4. Asignar permisos básicos al usuario
ALTER ROLE db_datareader ADD MEMBER usuario_farmacia;
ALTER ROLE db_datawriter ADD MEMBER usuario_farmacia;
GO

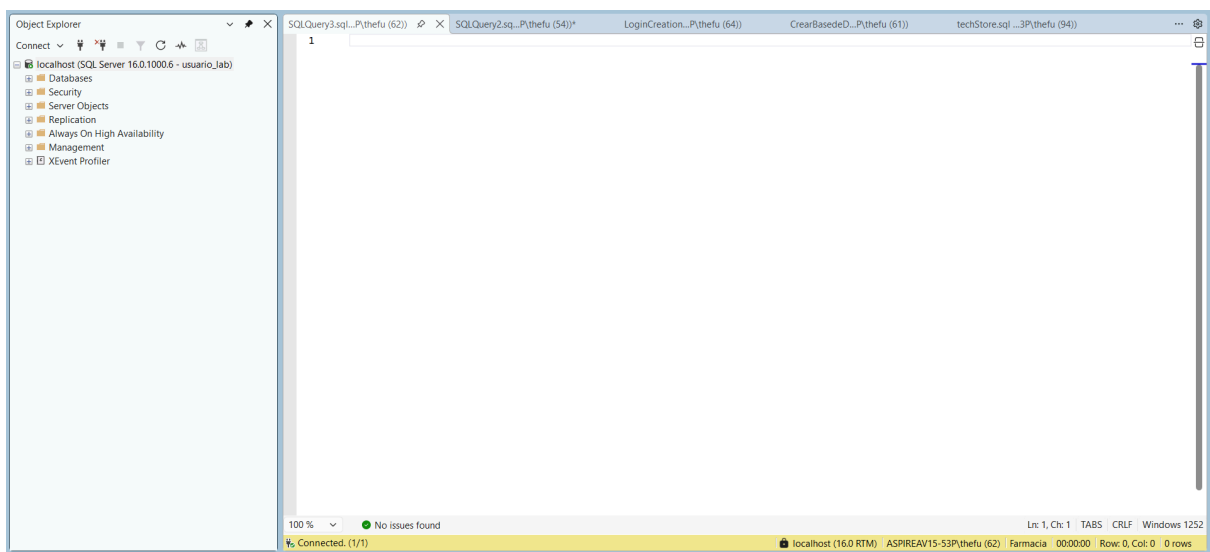
```

Después de creado, vamos a verificar si podemos acceder con este usuario a nuestra base de datos.

Usamos SQL server authentication:

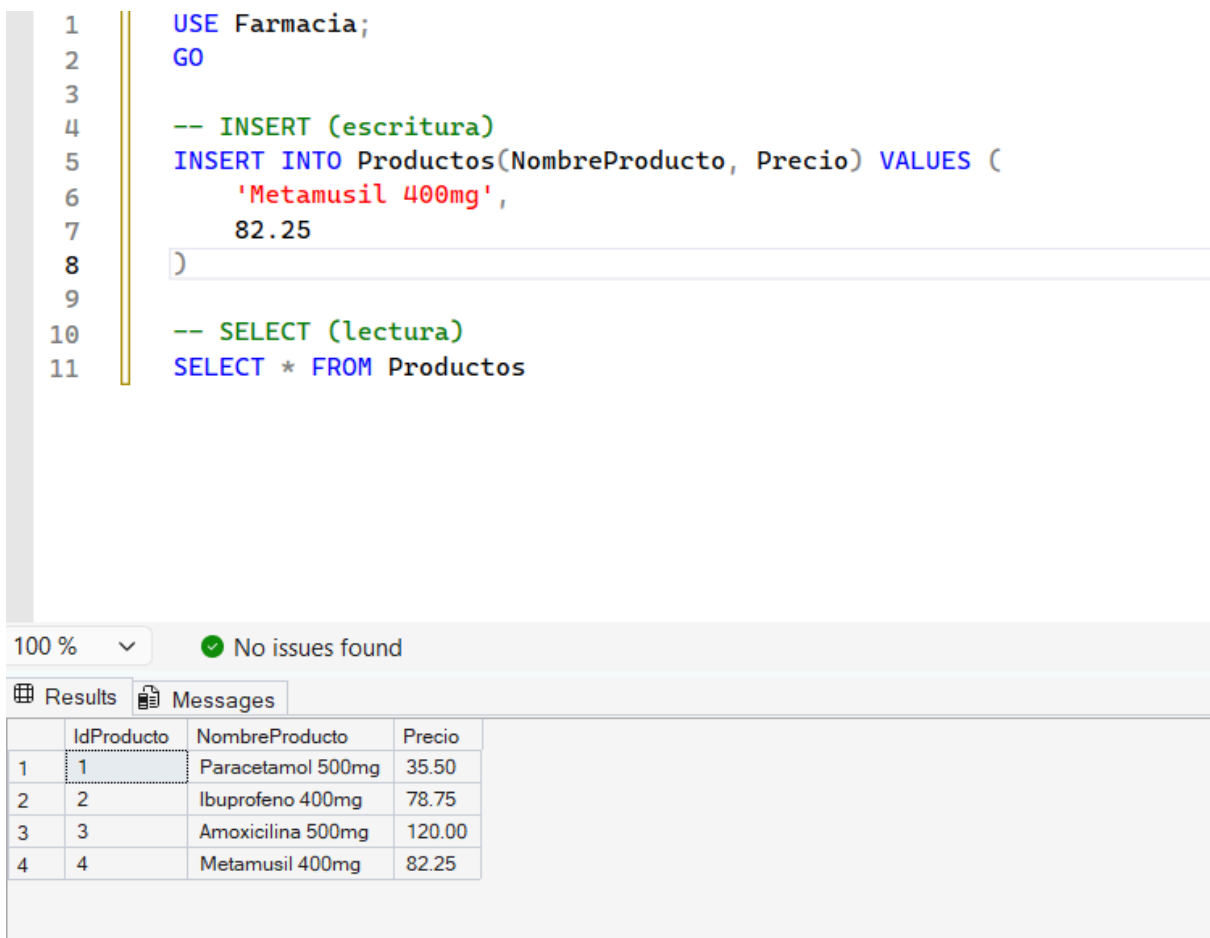


y procedemos a conectarnos



(Conexión exitosa)

Ahora correremos queries para ver si podemos escribir y leer de nuestra base de datos



Como podemos ver, el usuario asociado al login usuario_lab puede leer y escribir exitosamente a la base de datos creada.

Grupo 4: Roles predefinidos y permisos Actividades

- Asignar db_datareader.
- Probar restricción.
- Agregar db_datawriter.
- Verificar cambio de comportamiento.

Pautas para orientar solución

1. Forzar primero el error de permisos.
2. Usar:

ALTER ROLE db_datareader

ADD MEMBER usuario_lab

3. Comparar comportamiento antes/después y entregar resultados al Grupo 5.

Entregable

Evidencia del error y solución.

Primero desde el superuser removemos los permisos de usuario_farmacia

```

1      USE Farmacia;
2
3      -- Verificar que el usuario existe
4
5      SELECT name, type_desc
6      FROM sys.database_principals
7      WHERE name = 'usuario_farmacia';
8      GO
9
10     -- Removemos los permisos de usuario_farmacia
11
12     ALTER ROLE db_datareader
13     DROP MEMBER usuario_farmacia;
14
15     ALTER ROLE db_datawriter
16     DROP MEMBER usuario_farmacia;

```

0 % No issues found

Results		Messages
name	type_desc	
usuario_farmacia	SQL_USER	

Después probamos desde usuario farmacia un SELECT y un INSERT

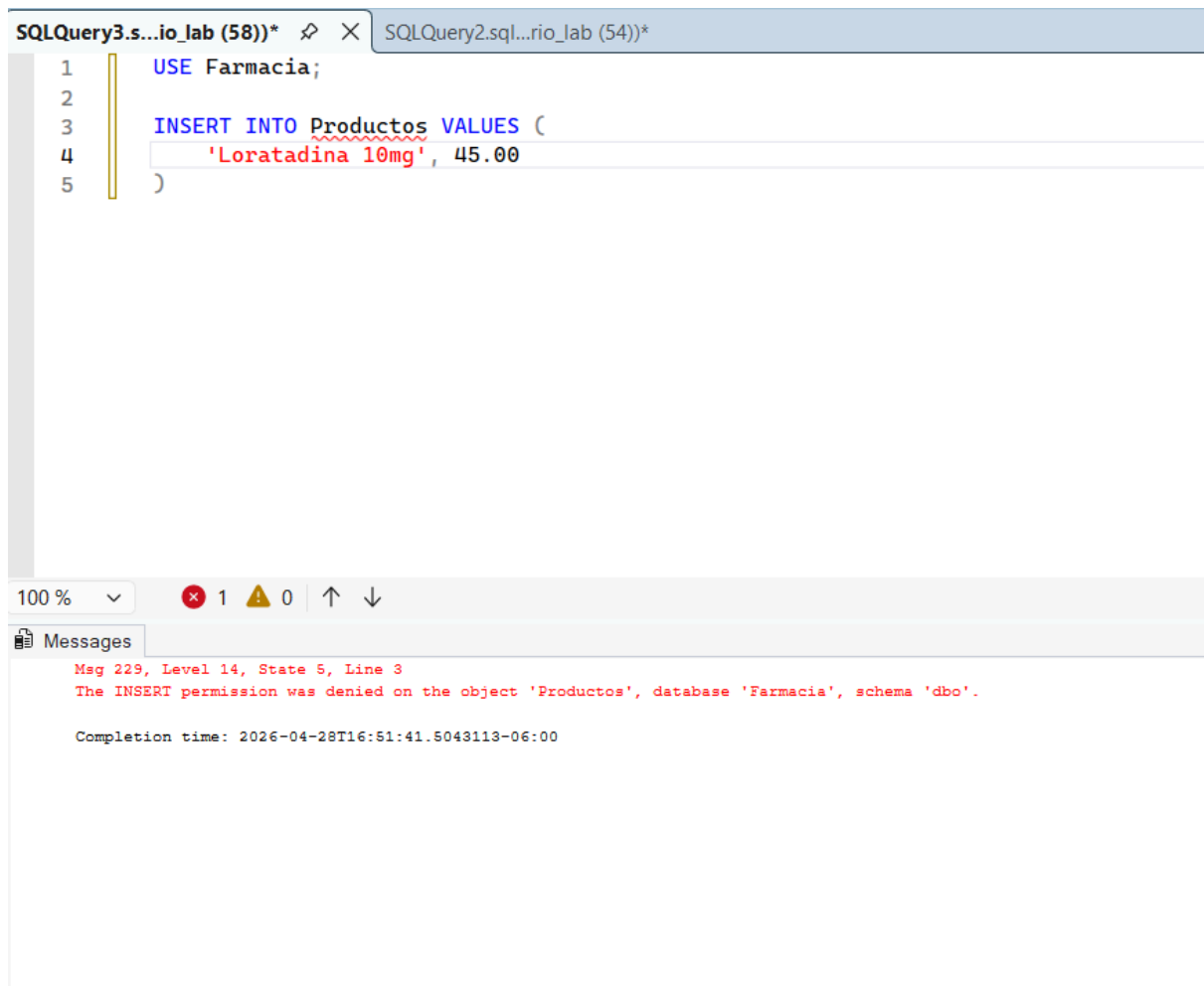
The screenshot shows a SQL Server Enterprise Manager window titled 'SQLQuery2.s...io_lab (54))*'. The query editor contains the following SQL code:

```
1 USE Farmacia;  
2  
3 SELECT * FROM Productos;
```

Below the query editor, the status bar indicates '100 %' zoom, '1' error, '0' warnings, and navigation arrows. The 'Messages' pane at the bottom displays the following error message:

```
Msg 229, Level 14, State 5, Line 3  
The SELECT permission was denied on the object 'Productos', database 'Farmacia', schema 'dbo'.  
  
Completion time: 2026-04-28T16:50:10.0154797-06:00
```

No se puede hacer el respectivo select ni tampoco podremos hacer el insert:



The screenshot shows a SQL Server Enterprise Manager window with two tabs: 'SQLQuery3.s...io_lab (58))*' and 'SQLQuery2.sql...rio_lab (54))*'. The active tab contains the following SQL code:

```
1  USE Farmacia;
2
3  INSERT INTO Productos VALUES (
4  'Loratadina 10mg', 45.00
5  )
```

Below the code editor, the 'Messages' pane displays an error message:

Msg 229, Level 14, State 5, Line 3
The INSERT permission was denied on the object 'Productos', database 'Farmacia', schema 'dbo'.
Completion time: 2026-04-28T16:51:41.5043113-06:00

Para devolverle los permisos, podemos aplicar lo mismo que antes pero a la inversa

```
1  USE Farmacia;
2
3  -- Verificar que el usuario existe
4
5  SELECT name, type_desc
6  FROM sys.database_principals
7  WHERE name = 'usuario_farmacia';
8  GO
9
10 -- Removemos los permisos de usuario_farmacia
11
12 ALTER ROLE db_datareader
13 ADD MEMBER usuario_farmacia;
14
15 ALTER ROLE db_datawriter
16 ADD MEMBER usuario_farmacia;
```

Ahora podemos ver que el Insert funciona

```
1  INSERT INTO Productos (NombreProducto, Precio)
2  VALUES ('Loratadina 10mg', 45.00);
3  GO
```

100 %   No issues found

Messages

(1 row affected)

Completion time: 2026-04-28T16:44:01.4615593-06:00

Y también el select

SQLQuery6.s...thefu (55))* RoleCreation....3P\thefu (73))

1 `SELECT * FROM Productos`

100 % No issues found

Results Messages

	IdProducto	NombreProducto	Precio
1	1	Paracetamol 500mg	35.50
2	2	Ibuprofeno 400mg	78.75
3	3	Amoxicilina 500mg	120.00
4	4	Metamusil 400mg	82.25
5	5	Loratadina 10mg	45.00

Grupo 5: Auditoría y Gobierno de Datos Actividades

- Revisar accesos.
- Identificar trazabilidad.
- Documentar permisos existentes.

Pautas para orientar solución

1. Utilizar:
sp_helpuser

2. Construir matriz usuario-permisos.
3. Entregar mini informe para el Grupo 6.

Entregable

Informe de auditoría.

Grupo 6: Recuperación, respaldo y retención Actividades

- Revisar recovery model.
- Ejecutar respaldo.
- Diseñar política de retención.

Pautas para orientar solución

1. Verificar modo de recuperación.
2. Probar:

BACKUP DATABASE Farmacia

TO DISK='C:\Backup\Farmacia.bak';

3. Entregar propuesta de respaldo al Grupo 7.

Entregable

Evidencia + política de retención.

Ejecutamos la siguiente query para poder ver el recovery model de la base de datos

```
1 USE master;
2 GO
3
4 SELECT
5     name AS BaseDeDatos,
6     recovery_model_desc AS ModeloRecuperacion
7 FROM sys.databases
8 WHERE name = 'Farmacia';
9 GO
```

100 % ▾

✓ No issues found

Results Messages

	BaseDeDatos	ModeloRecuperacion
1	Farmacia	FULL

FULL → permite recuperación completa (mejor para producción) a diferencia de SIMPLE.

Ahora para crear el backup completo de la base de datos hacemos uso de este query

```
SQLQuery7.s...thefu (56))*  SQLQuery6.sq...P\thefu (55))*  RoleCreation....3P\thefu (7)
1  BACKUP DATABASE Farmacia
2  TO DISK = 'C:\DATABASES\BACKUPS\Farmacia.bak'
3  WITH
4      FORMAT,
5      INIT,
6      NAME = 'Backup Completo Farmacia';
7  GO
```

100 % No issues found

Messages

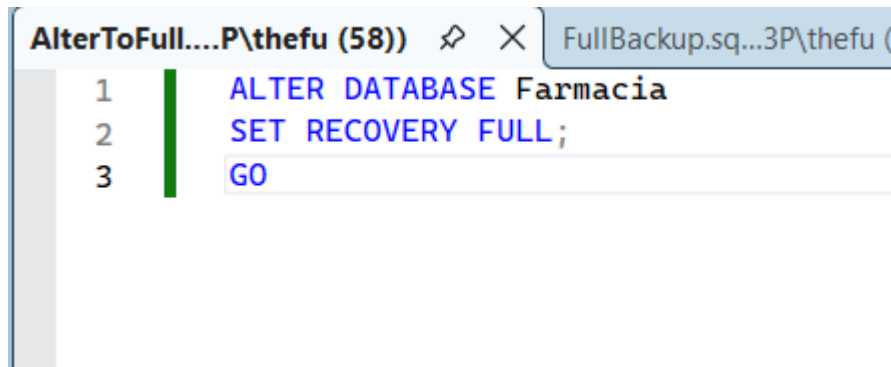
Processed 568 pages for database 'Farmacia', file 'Farmacia' on file 1.
Processed 1 pages for database 'Farmacia', file 'Farmacia_log' on file 1.
BACKUP DATABASE successfully processed 569 pages in 0.049 seconds (90.591 MB/sec).

Completion time: 2026-04-28T17:09:07.1739593-06:00

Podemos verificar su ubicación:

This PC > Local Disk (C:) > DATABASES > BACKUPS				
Sort View ...				
Name	Date modified	Type	Size	
Farmacia.bak	28/04/2026 05:09 p. m.	BAK File	4,633 KB	

(Bonus: en el caso de que la base de datos esté en SIMPLE, se puede poner a FULL con esta query)



```
AlterToFull....P\thefu (58))  FullBackup.sq...3P\thefu (
1  ALTER DATABASE Farmacia
2  SET RECOVERY FULL;
3  GO
```

Política de retención:

Se propone una estrategia basada en respaldos completos diarios, diferenciales cada 6 horas y respaldos de log cada 30 minutos. Los respaldos diarios se conservarán por 7 días, los semanales por 1 mes y los mensuales por 6 meses. Además, se recomienda almacenar copias en ubicaciones externas y realizar pruebas periódicas de restauración para garantizar la integridad de los datos.

Grupo 7: Documentación y metadatos Actividades

- Documentar la estructura de la base de datos y la tabla construida por los grupos anteriores.
- Utilizar herramientas de ayuda del gestor para consultar metadatos.
- Elaborar un diccionario de datos básico y un documento de especificaciones.

Pautas para orientar solución

1. Utilizar el procedimiento del sistema: sp_help Productos
2. Documentar información como:
 - Nombre de campos
 - Tipos de datos
 - Longitud
 - Llaves primarias
 - Restricciones y propiedades identificadas
3. Elaborar un documento sencillo de especificaciones y entregarlo como insumo para el Grupo

Entregable

Documento sencillo de especificaciones y diccionario de datos con evidencias.

Documentación

```
-- Documentación
-- Procedimiento del sistema
USE Farmacia;
GO
```

Esta consulta se utiliza para indicar que todas las operaciones posteriores se realizarán dentro de la base de datos Farmacia. El comando USE Farmacia cambia el contexto de trabajo hacia esa base de datos, evitando que las consultas se ejecuten accidentalmente en otra base de datos del servidor. La instrucción GO separa y ejecuta el bloque de código en SQL Server Management Studio.

```
EXEC sp_help Productos;
GO
```

49
50
51
52
53
54
55
56
57

```
EXEC sp_help Productos;  
GO  
  
-- Consulta de los datos de la tabla  
SELECT *  
FROM Productos;  
GO
```

79 %

No se encontraron problemas.

Resultados

Mensajes

	Name	Owner	Type	Created_datetime
1	Productos	dbo	user table	2026-04-28 16:06:17.860

	Column_name	Type	Computed	Length	Prec	Scale	Nullable	TrimTrailingBlanks	FixedLenNullInSource	Collation
1	IdProducto	int	no	4	10	0	no	(n/a)	(n/a)	NULL
2	NombreProducto	varchar	no	100			no	no	no	Modern_Spanish_CI_AS
3	Precio	decimal	no	9	10	2	no	(n/a)	(n/a)	NULL

	Identity	Seed	Increment	Not For Replication
1	IdProducto	1	1	0

	RowGuidCol
1	No rowguidcol column defined.

	Data_located_on_filegroup
1	PRIMARY

	index_name	index_description	index_keys
1	PK_Producto__098892109D77E30F	clustered, unique, primary key located on PRIMARY	IdProducto

	constraint_type	constraint_name	delete_action	update_action	status_enabled	status_for_replication	constraint_keys
1	PRIMARY KEY (clustered)	PK_Producto__098892109D77E30F	(n/a)	(n/a)	(n/a)	(n/a)	IdProducto

Esta consulta utiliza el procedimiento del sistema sp_help para obtener información general y técnica sobre la tabla Productos. A través de este procedimiento se pueden consultar los metadatos de la tabla, como el nombre de los campos, tipos de datos, longitudes, si los campos permiten valores nulos, restricciones, índices y claves asociadas. Esta consulta es importante porque permite documentar la estructura de la tabla sin tener que revisar manualmente cada propiedad desde el Explorador de Objetos de SQL Server.

En la primera sección se muestra la información general de la tabla. El nombre del objeto es Productos, pertenece al esquema dbo, su tipo es user table y fue creada el 2026-04-28 a las

16:06:17.860. Esto indica que Productos es una tabla creada por el usuario y no una tabla interna del sistema.

En la sección de columnas se observa que la tabla Productos está formada por tres campos: IdProducto, NombreProducto y Precio. El campo IdProducto es de tipo int, no permite valores nulos y funciona como identificador principal de cada producto. El campo NombreProducto es de tipo varchar con una longitud máxima de 100 caracteres y tampoco permite valores nulos, por lo que cada producto debe tener un nombre registrado. El campo Precio es de tipo decimal con precisión 10 y escala 2, lo que permite almacenar precios con dos decimales, y también es obligatorio.

En la sección Identity se confirma que el campo IdProducto tiene la propiedad IDENTITY. Esto significa que SQL Server genera automáticamente su valor, iniciando en 1 y aumentando de uno en uno por cada nuevo registro insertado. Esta propiedad evita que el usuario tenga que ingresar manualmente el código de cada producto.

También se muestra que la tabla no tiene una columna RowGuidCol definida. Esto no representa un problema, ya que ese tipo de columna se utiliza en casos más avanzados, como replicación o identificación global de registros. Para esta tabla, el campo IdProducto cumple correctamente la función de identificador único.

En la sección de almacenamiento se observa que los datos de la tabla están ubicados en el filegroup PRIMARY. Esto significa que la tabla Productos se almacena en el grupo de archivos principal de la base de datos, lo cual es normal en una base de datos sencilla como esta.

En la sección de índices se identifica un índice asociado a la clave primaria de la tabla. Este índice es clustered, unique y primary key, y está aplicado sobre el campo IdProducto. Esto significa que IdProducto no solo identifica de forma única a cada producto, sino que también organiza físicamente los registros de la tabla.

Finalmente, en la sección de restricciones se confirma que existe una restricción de tipo PRIMARY KEY clustered sobre el campo IdProducto. Esta restricción asegura que cada producto tenga un identificador único y que no existan registros duplicados con el mismo IdProducto.

```
-- Consulta de los datos de la tabla
✓ SELECT *
  FROM Productos;
- GO
```

52	-- Consulta de los datos de la tabla
53	SELECT *
54	FROM Productos;
55	GO

79 % No se encontraron problemas.

Resultados Mensajes

	IdProducto	NombreProducto	Precio
1	1	Paracetamol 500mg	35.50
2	2	Ibuprofeno 400mg	78.75
3	3	Amoxicilina 500mg	120.00

Esta consulta muestra todos los registros almacenados en la tabla Productos. El símbolo * indica que se desean visualizar todas las columnas de la tabla, que en este caso son IdProducto, NombreProducto y Precio. Esta consulta sirve como evidencia de que la tabla fue creada correctamente y que contiene los tres registros de ejemplo insertados: Paracetamol 500mg, Ibuprofeno 400mg y Amoxicilina 500mg.

```
-- Llave primaria
SELECT
    KU.TABLE_NAME AS NombreTabla,
    KU.COLUMN_NAME AS CampoClavePrimaria,
    TC.CONSTRAINT_NAME AS NombreRestriccion
FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS AS TC
INNER JOIN INFORMATION_SCHEMA.KEY_COLUMN_USAGE AS KU
    ON TC.CONSTRAINT_NAME = KU.CONSTRAINT_NAME
WHERE TC.CONSTRAINT_TYPE = 'PRIMARY KEY'
    AND KU.TABLE_NAME = 'Productos';
GO
```

57	-- Llave primaria
58	SELECT
59	KU.TABLE_NAME AS NombreTabla,
60	KU.COLUMN_NAME AS CampoClavePrimaria,
61	TC.CONSTRAINT_NAME AS NombreRestriccion
62	FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS AS TC
63	INNER JOIN INFORMATION_SCHEMA.KEY_COLUMN_USAGE AS KU
64	ON TC.CONSTRAINT_NAME = KU.CONSTRAINT_NAME
65	WHERE TC.CONSTRAINT_TYPE = 'PRIMARY KEY'
66	AND KU.TABLE_NAME = 'Productos';
67	GO

9 % No se encontraron problemas.

Resultados Mensajes

	NombreTabla	CampoClavePrimaria	NombreRestriccion
1	Productos	IdProducto	PK__Producto__098892109D77E30F

Esta consulta permite identificar cuál es la clave primaria de la tabla Productos. Para ello, utiliza las vistas INFORMATION_SCHEMA.TABLE_CONSTRAINTS e INFORMATION_SCHEMA.KEY_COLUMN_USAGE, las cuales almacenan información

sobre las restricciones y los campos relacionados con ellas. En este caso, la consulta debe mostrar que el campo IdProducto es la clave primaria de la tabla.

En la consulta de la clave primaria, el INNER JOIN se utiliza para relacionar la información de las restricciones con las columnas que forman parte de esas restricciones. La vista TABLE_CONSTRAINTS permite conocer el tipo de restricción, mientras que KEY_COLUMN_USAGE permite saber qué columna está asociada a dicha restricción. Gracias a esta unión, se puede identificar correctamente que IdProducto pertenece a una restricción de tipo clave primaria.

La condición WHERE TC.CONSTRAINT_TYPE = 'PRIMARY KEY' filtra los resultados para mostrar únicamente las restricciones de tipo clave primaria. Esto evita que aparezcan otros tipos de restricciones, como claves únicas o claves foráneas. Además, la condición AND KU.TABLE_NAME = 'Productos' limita la búsqueda únicamente a la tabla Productos, para que el resultado sea específico de la tabla que se está documentando.

```
-- Restricciones de la tabla
SELECT
    tc.TABLE_NAME AS NombreTabla,
    tc.CONSTRAINT_NAME AS NombreRestriccion,
    tc.CONSTRAINT_TYPE AS TipoRestriccion
FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS tc
WHERE tc.TABLE_NAME = 'Productos';
GO
```

```
-- Llave primaria
57
58 SELECT
59     KU.TABLE_NAME AS NombreTabla,
60     KU.COLUMN_NAME AS CampoClavePrimaria,
61     TC.CONSTRAINT_NAME AS NombreRestriccion
62 FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS AS TC
63 INNER JOIN INFORMATION_SCHEMA.KEY_COLUMN_USAGE AS KU
64     ON TC.CONSTRAINT_NAME = KU.CONSTRAINT_NAME
65 WHERE TC.CONSTRAINT_TYPE = 'PRIMARY KEY'
66     AND KU.TABLE_NAME = 'Productos';
67 GO
```

3 % No se encontraron problemas.

Resultados Mensajes

	NombreTabla	CampoClavePrimaria	NombreRestriccion
1	Productos	IdProducto	PK__Producto__098892109D77E30F

Esta consulta permite visualizar las restricciones principales asociadas a la tabla Productos. El resultado muestra el nombre de la tabla, el nombre de la restricción y el tipo de restricción existente. En este caso, la restricción más importante es la clave primaria, la cual garantiza que cada producto tenga un identificador único mediante el campo IdProducto.

La restricción PRIMARY KEY aplicada al campo IdProducto asegura que cada registro de la tabla tenga un valor único e irrepetible. Además, una clave primaria no permite valores nulos, por lo que siempre debe existir un identificador para cada producto. Esta restricción es fundamental para mantener la integridad de la tabla y facilitar la identificación de cada producto almacenado.

El campo IdProducto fue definido con la propiedad IDENTITY(1,1), lo que significa que SQL Server genera automáticamente su valor. El primer número indica que la numeración inicia en 1, y el segundo número indica que aumenta de uno en uno por cada nuevo registro insertado. Esta propiedad evita que el usuario tenga que escribir manualmente el identificador del producto.

Los campos NombreProducto y Precio fueron definidos con la restricción NOT NULL, lo que significa que no pueden quedar vacíos al momento de insertar un producto. Esta restricción mejora la integridad de los datos, ya que evita registrar productos sin nombre o sin precio. De esta forma, la tabla mantiene información completa y útil para el sistema.

La tabla Productos pertenece a la base de datos Farmacia y tiene como finalidad almacenar información básica sobre los productos farmacéuticos. Su estructura está compuesta por tres campos: IdProducto, NombreProducto y Precio. El campo IdProducto identifica de forma única a cada producto, NombreProducto almacena el nombre del medicamento o producto, y Precio registra el valor económico del producto con dos decimales.

La consulta SELECT * FROM Productos permite comprobar que la tabla contiene registros almacenados correctamente. Los datos insertados funcionan como ejemplos para validar que la estructura de la tabla permite guardar productos con su respectivo nombre y precio. Esta evidencia es importante porque demuestra que la tabla no solo fue creada, sino que también puede almacenar información de manera funcional.

Restricciones y propiedades identificadas

Campo	Restricciones identificadas	Propiedades identificadas
IdProducto	PRIMARY KEY, NOT NULL	IDENTITY(1,1), índice clustered, unique, ubicado en PRIMARY
NombreProducto	NOT NULL	VARCHAR(100), collation Modern_Spanish_CI_AS
Precio	NOT NULL	DECIMAL(10,2), permite dos decimales

Diccionario de datos

Campo	Tipo de dato	Longitud/precisión	Permite nulos	Propiedad o restricción	Descripción
IdProducto	INT	4 bytes	No	IDENTITY(1,1), PRIMARY KEY	Identificador único de cada producto. Se genera automáticamente iniciando en 1 y aumentando de uno en uno.
NombreProducto	VARCHAR	100 caracteres	No	NOT NULL	Almacena el nombre del producto farmacéutico. Es un campo obligatorio.
Precio	DECIMAL	DECIMAL(10,2)	No	NOT NULL	Almacena el precio del producto con dos decimales. Es un campo obligatorio.

Grupo 8: DevOps y control de versiones Actividades

- Simular control de versiones sobre scripts SQL desarrollados en la práctica.
- Registrar cambios realizados sobre estructura y seguridad.
- Elaborar una bitácora de trazabilidad de configuraciones.

Pautas para orientar solución

1. Crear y documentar versiones de scripts, por ejemplo:
 - v1: Creación de base de datos y tabla
 - v2: Modificaciones o ajustes de estructura (ejemplo: agregar campo)
 - v3: Seguridad y permisos
2. Llevar una bitácora de cambios indicando:
 - Cambio realizado
 - Motivo del cambio
 - Versión asociada
3. Entregar historial de cambios como insumo para el Grupo 9.

Entregable

Bitácora de cambios y scripts versionados.

```
-- v1_creacion_base_tabla.sql

IF NOT EXISTS (
    SELECT name
    FROM sys.databases
    WHERE name = 'Farmacia'
)
BEGIN
    CREATE DATABASE Farmacia;
END;
GO

USE Farmacia;
GO

IF NOT EXISTS (
    SELECT *
    FROM sys.tables
    WHERE name = 'Productos'
)
BEGIN
    CREATE TABLE Productos (
        IdProducto INT IDENTITY(1,1) PRIMARY KEY,
        NombreProducto VARCHAR(100) NOT NULL,
        Precio DECIMAL(10,2) NOT NULL
    );
END;
GO
```

Esta primera versión crea la base de datos Farmacia y la tabla Productos. la tabla contiene los campos básicos para registrar productos farmacéuticos: identificador, nombre y precio

```
-- v2_datos_consultas.sql

USE Farmacia;
GO

INSERT INTO Productos (NombreProducto, Precio)
VALUES
    ('Paracetamol 500mg', 35.50),
    ('Ibuprofeno 400mg', 78.75),
    ('Amoxicilina 500mg', 120.00);
GO

SELECT *
FROM Productos;
GO

SELECT IdProducto, NombreProducto, Precio
FROM Productos
WHERE Precio > 50;
GO

SELECT IdProducto, NombreProducto, Precio
FROM Productos
ORDER BY Precio ASC;
GO
```

En esta versión se agregan registros de prueba y se ejecutan consultas básicas para comprobar el funcionamiento de la tabla. También se validan filtros por precio y ordenamiento ascendente.

```
-- v3_seguridad_permisos.sql

USE master;
GO

CREATE LOGIN usuario_lab
WITH PASSWORD = 'lab12345*';
GO

USE Farmacia;
GO

CREATE USER usuario_farmacia
FOR LOGIN usuario_lab;
GO

ALTER ROLE db_datareader ADD MEMBER usuario_farmacia;
ALTER ROLE db_datawriter ADD MEMBER usuario_farmacia;
GO

SELECT name, type_desc
FROM sys.database_principals
WHERE name = 'usuario_farmacia';
GO
```

Esta versión incorpora configuración de seguridad. Se crea un login a nivel de servidor y un usuario asociado dentro de la base de datos Farmacia. Luego se asignan permisos de lectura y escritura mediante los roles db_datareader y db_datawriter.

```
-- v4_prueba_y_retiro_permisos.sql

USE Farmacia;
GO

INSERT INTO Productos (NombreProducto, Precio)
VALUES ('Metamusal 400mg', 82.25);
GO

SELECT *
FROM Productos;
GO

ALTER ROLE db_datareader DROP MEMBER usuario_farmacia;
ALTER ROLE db_datawriter DROP MEMBER usuario_farmacia;
GO
```

En esta versión se realiza una prueba de inserción para validar los permisos asignados al usuario. Posteriormente, se retiran los roles de lectura y escritura como parte del control de accesos.

Versión	Cambio realizado	Motivo del cambio
versión 1	Creación de la base de datos Farmacia y tabla Productos	Establecer la estructura inicial del sistema

versión 2	Insertión de datos y consultas básicas	Validar que la tabla funcione correctamente
versión 3	Creación de login, usuario y asignación de roles	Implementar control de acceso y seguridad
versión 4	Prueba de inserción y retiro de permisos	Verificar permisos y aplicar control de acceso

Grupo 9: Optimización de base de datos Actividades

- Analizar oportunidades básicas de optimización sobre la solución construida.
- Identificar buenas prácticas de diseño, rendimiento e integridad.
- Elaborar una propuesta técnica de mejora.

Pautas para orientar solución

1. Analizar posibles mejoras como:
 - Uso de índices
 - Validaciones mediante restricciones
 - Mejoras de diseño de tablas
2. Formular al menos tres recomendaciones justificadas para optimizar rendimiento o integridad.
3. Entregar propuesta técnica al Grupo 10 para relacionarla con escalamiento distribuido.

Propuesta técnica de optimización para la base de datos Farmacia

Después de revisar la base de datos Farmacia y la tabla Productos, se identificaron oportunidades básicas de mejora orientadas al rendimiento, la integridad de los datos y el diseño de la tabla. Actualmente la tabla cuenta con una llave primaria en IdProducto, pero puede fortalecerse mediante restricciones adicionales e índices que ayuden a controlar mejor la información registrada.

1. Agregar restricción para evitar precios inválidos

Actualmente el campo Precio permite registrar valores numéricos, pero no impide que se inserten precios en cero o negativos. Para mejorar la integridad de los datos, se recomienda agregar una restricción CHECK.

```
USE Farmacia;
```

```
GO
```

```
ALTER TABLE Productos
```

```
ADD CONSTRAINT CK_Productos_Precio_Positive
```

```
CHECK (Precio > 0);
```

```
GO
```

Esta mejora garantiza que todos los productos registrados tengan un precio válido mayor que cero, evitando errores en consultas, reportes o procesos de venta.

2. Evitar nombres de productos repetidos

Se recomienda agregar una restricción UNIQUE al campo NombreProducto para evitar que el mismo producto sea registrado más de una vez.

```
USE Farmacia;
```

```
GO
```

```
ALTER TABLE Productos
```

```
ADD CONSTRAINT UQ_Productos_NombreProducto
```

```
UNIQUE (NombreProducto);
```

```
GO
```

Esta restricción mejora la calidad de los datos, ya que evita duplicidad de productos como “Paracetamol 500mg” registrado varias veces con precios diferentes.

3. Crear índice para búsquedas por precio

Como ya existe una consulta que busca productos mayores a cierto precio, se recomienda crear un índice sobre el campo Precio para mejorar el rendimiento de consultas con filtros.

```
USE Farmacia;  
  
GO  
  
CREATE INDEX IX_Productos_Precio  
  
ON Productos (Precio);  
  
GO
```

Este índice permite que SQL Server localice más rápido los productos cuando se realizan consultas como **WHERE Precio > 50**, especialmente si la tabla crece con muchos registros.

Consulta de validación

Después de aplicar las mejoras, se puede comprobar que las restricciones e índices fueron creados correctamente con:

```
USE Farmacia;  
  
GO  
  
SELECT  
  
    name AS NombreObjeto,  
  
    type_desc AS TipoObjeto  
  
FROM sys.objects  
  
WHERE name IN (  
  
    'CK_Productos_Precio_Positive',  
  
    'UQ_Productos_NombreProducto'  
  
);  
  
GO  
  
SELECT  
  
    name AS NombreIndice
```

```
FROM sys.indexes  
  
WHERE object_id = OBJECT_ID('Productos');  
  
GO
```

Recomendaciones finales

Se recomienda mantener el uso de la llave primaria IdProducto, ya que permite identificar cada producto de forma única. También se recomienda aplicar restricciones como CHECK y UNIQUE para evitar datos incorrectos o duplicados. Finalmente, se sugiere utilizar índices en columnas utilizadas frecuentemente en búsquedas, como Precio, ya que esto mejora el rendimiento cuando la cantidad de registros aumenta.

Con estas mejoras, la base de datos queda mejor preparada para crecer y para ser utilizada posteriormente por el Grupo 10 en un posible escenario de escalamiento o distribución.

Entregable

Propuesta de optimización y defensa oral breve.

Grupo 10: Arquitectura distribuida y nodos Actividades

- Analizar de forma introductoria una arquitectura de bases de datos distribuidas.
- Diseñar un esquema conceptual de dos nodos (por ejemplo, dos sucursales).
- Documentar una propuesta básica de distribución de datos.

Pautas para orientar solución

1. Representar gráficamente dos nodos distribuidos y su posible comunicación.
2. Explicar conceptualmente:
 - Intercambio de datos entre nodos
 - Sincronización básica
 - Desafíos de consistencia
3. Relacionar la solución desarrollada con una propuesta de escalamiento distribuido.

Entregable

Diagrama conceptual de nodos y explicación técnica.

Flujo Secuencial de la Práctica de Laboratorio

1. Recorrido
2. Construcción
3. Seguridad
4. Permisos
5. Auditoría
6. Recuperación
7. Documentación
8. DevOps
9. Optimización
10. Distribución

Propuesta de arquitectura distribuida

En esta fase se propone una arquitectura distribuida básica para la base de datos Farmacia, construida en las fases anteriores de la práctica. Esta propuesta representa cómo la solución podría escalar si la empresa farmacéutica necesitara operar en más de una sucursal.

Actualmente, la base de datos contiene la tabla Productos, compuesta por los campos IdProducto, NombreProducto y Precio. Esta tabla almacena información básica de los productos farmacéuticos y puede utilizarse como catálogo principal para diferentes sucursales.

Para esta propuesta se plantea una arquitectura con dos nodos:

Nodo 1: Sucursal Managua

Este nodo representa la sucursal principal. Contiene una instancia de SQL Server con la base de datos Farmacia y la tabla Productos. Desde este nodo se pueden registrar, consultar y actualizar productos.

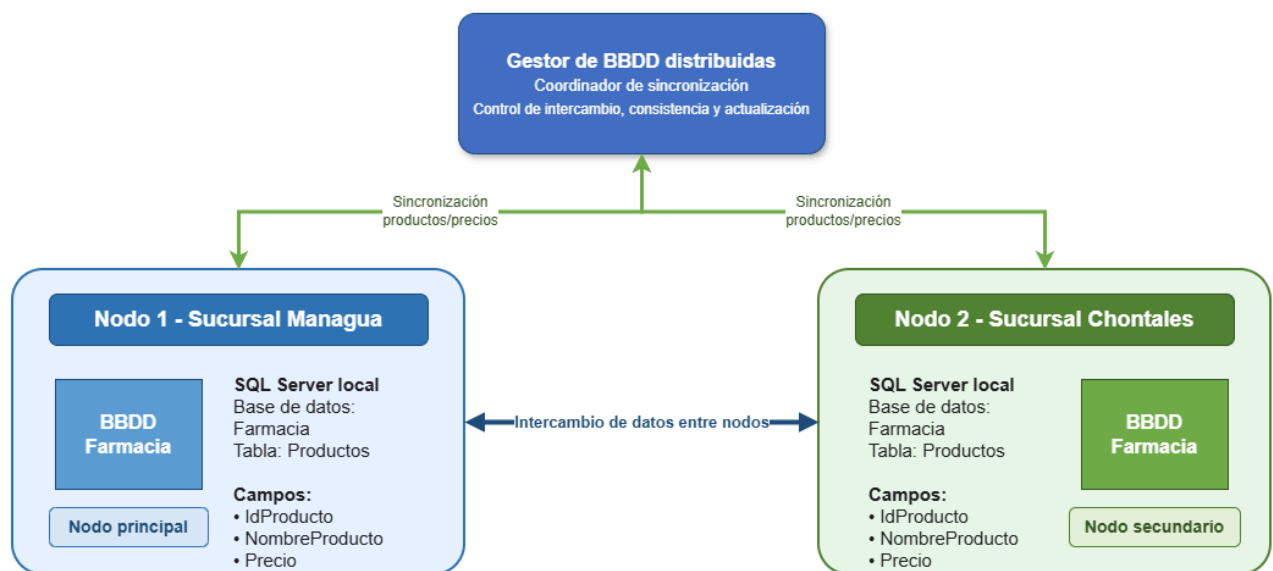
Nodo 2: Sucursal Chontales

Este nodo representa una sucursal secundaria. También cuenta con una instancia de SQL Server con la base de datos Farmacia y la tabla Productos. Este nodo recibe la información

actualizada del catálogo de productos para mantener los datos alineados con la sucursal principal.

La comunicación entre ambos nodos permite que los datos principales de productos se mantengan actualizados. Por ejemplo, si en la sucursal Managua se modifica el precio de un medicamento o se agrega un nuevo producto, dicho cambio debe sincronizarse posteriormente con la sucursal Chontales para evitar diferencias en la información.

Esquema conceptual de dos nodos



Intercambio de datos entre nodos

El intercambio de datos consiste en compartir la información relevante de la tabla Productos entre ambas sucursales. En este caso, los datos principales que se intercambian son el identificador del producto, el nombre del producto farmacéutico y su precio.

Este intercambio permite que las sucursales trabajen con el mismo catálogo de productos. De esta manera, se evita que existan diferencias entre los productos disponibles o entre los precios registrados en cada sucursal.

Por ejemplo, si la sucursal Managua agrega un nuevo producto llamado Loratadina 10mg, la sucursal Chontales debe recibir esa información durante el proceso de sincronización. Lo mismo ocurre si se actualiza el precio de un medicamento existente.

Sincronización básica

La sincronización básica puede realizarse de forma periódica entre los nodos. En esta propuesta, la sucursal Managua puede funcionar como nodo principal y la sucursal Chontales como nodo secundario.

El proceso de sincronización podría realizarse de la siguiente manera:

1. La sucursal Managua registra o actualiza productos en la base de datos Farmacia.
2. Los cambios quedan guardados en la tabla Productos.
3. La información actualizada se envía a la sucursal Chontales.
4. La sucursal Chontales actualiza su propia tabla Productos.
5. Ambas sucursales quedan con información equivalente.

En un entorno real, podría implementarse mediante replicación de SQL Server, respaldos programados, scripts automatizados o servicios de integración de datos.

Desafíos de consistencia

Uno de los principales desafíos de una base de datos distribuida es mantener la consistencia de la información. Si una sucursal actualiza el precio de un producto y la otra no recibe el cambio a tiempo, podrían existir diferencias entre los datos de ambos nodos.

También pueden presentarse conflictos si dos sucursales modifican el mismo producto al mismo tiempo. Por ejemplo, si Managua cambia el precio del Paracetamol y Chontales también modifica ese mismo precio antes de sincronizarse, el sistema debe definir cuál cambio tendrá prioridad.

Otro desafío importante es la disponibilidad de la red. Si la comunicación entre sucursales falla, cada nodo podría seguir trabajando de forma local, pero los cambios quedarían pendientes de sincronización hasta que la conexión se restablezca.

Por esta razón, es necesario definir reglas claras para la sincronización, la resolución de conflictos y la actualización de los datos compartidos.

Relación con la solución desarrollada

La propuesta distribuida se relaciona directamente con la base de datos construida durante la práctica, ya que utiliza la base Farmacia y la tabla Productos como punto de partida. Esta tabla fue creada, documentada y optimizada en las fases anteriores, por lo que puede ser utilizada como catálogo compartido entre sucursales.

Además, las mejoras propuestas en la fase de optimización, como el uso de restricciones e índices, ayudan a preparar la base de datos para un escenario de crecimiento. Las restricciones permiten mantener la integridad de los datos, mientras que los índices pueden mejorar el rendimiento de las consultas cuando la cantidad de productos aumente.

De esta forma, la arquitectura distribuida permite visualizar cómo una base de datos sencilla puede escalar hacia un modelo con varios nodos, manteniendo comunicación, sincronización y control de la información entre sucursales.

Estrategia de trabajo

Cada grupo deberá:

- Recibir insumos del grupo anterior.
- Desarrollar su fase.
- Entregar evidencias reutilizables al siguiente grupo.

Se simula una cadena colaborativa tipo consultoría/DevOps.